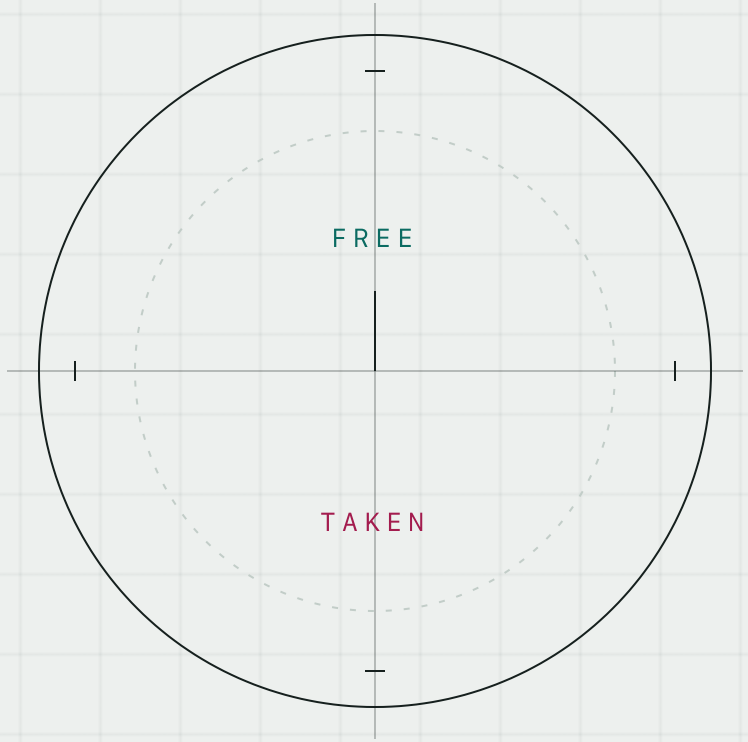
PROJECT

Shared Laboratory Equipment Booking System

A console register for who has which instrument, and when it comes back.

+ Arjun Tandon
+ Ansh Jaiswal
+ Priyanshu Sharma
+ Lakshit Verma



One place to borrow and return lab equipment.

Sign in as a student, a teacher or the lab admin. The list shows what is free. Borrowing marks the item taken and sets a due day. Returning it closes the record and charges for any delay.



STEP ONE

Sign in

The role sets the limits.



STEP TWO

See what is free

Every item shows free or taken.



STEP THREE

Borrow

The due day is fixed on the spot.



STEP FOUR

Return

The fine is charged if late.

Every action prints one line that says what happened and why. A refused borrow explains the reason.

Equipment and people are built the same way.

EQUIPMENT

Equipment

id, name, brand, free or taken, borrow count

- **Microscope** — magnification
- **Centrifuge** — top speed
- **Spectrometer** — range

PEOPLE

User

id, name, role rules

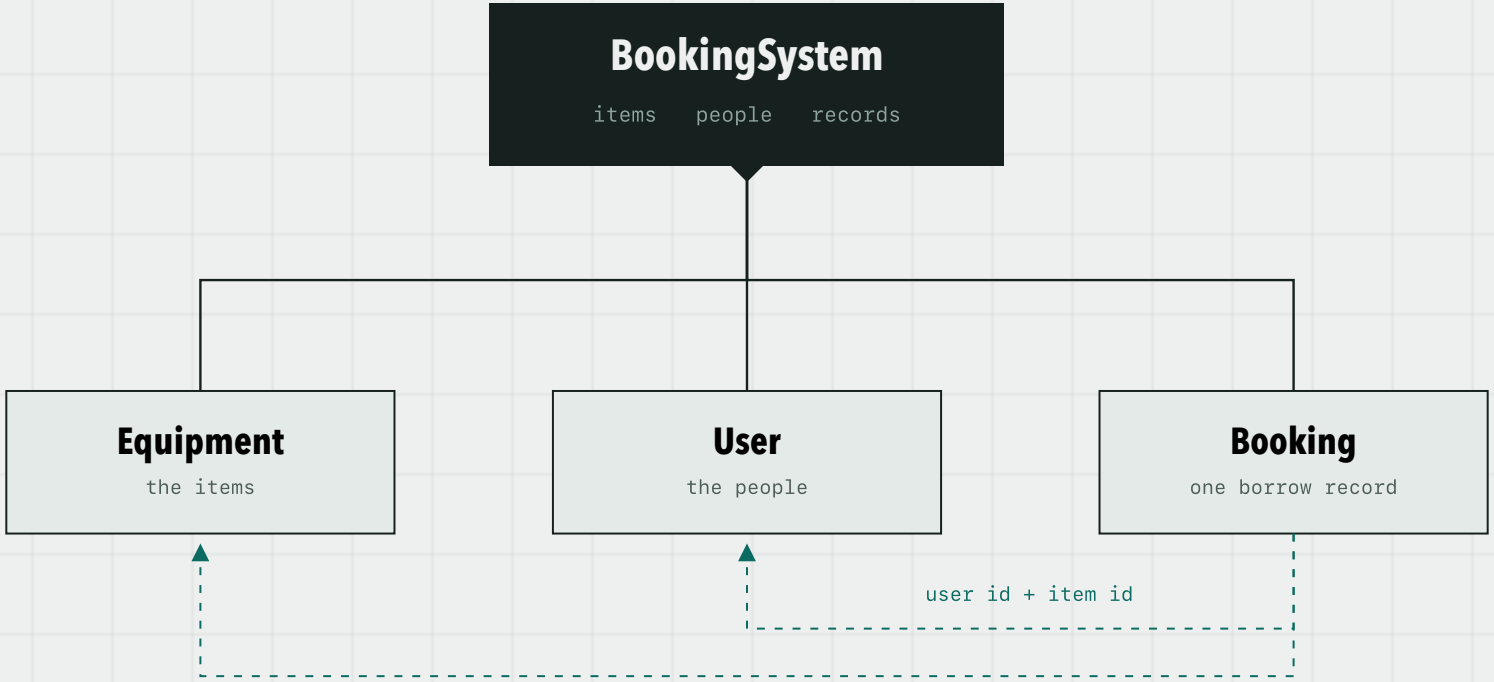
- **Student** — year 1, 2 or 3
- **Faculty** — 14 day loan
- **Lab admin** — 30 day loan

The base holds what every type shares. A child adds the rest. Only the child decides its own limit.

ONE ITEM IN MEMORY

101 **Olympus CX23** Microscope 1000x Olympus FREE

The manager owns the lists. A booking stores two ids.



OWNS

BookingSystem holds the items, the people and the records, and frees them when it closes.

LINKS BY ID

A Booking keeps a user id and an item id. Deleting an item cannot leave a broken link.

ONE WAY IN

All changes go through BookingSystem. Nothing edits the lists directly.

A borrow writes one record and marks the item taken. A return closes the record and frees the item.

Four ideas, four snippets.

```
class Equipment {
private:
    int id;
    bool available;      // hidden
public:
    bool isAvailable() const;
    void markBorrowed();  // only way
};
```

Encapsulation

State is private. Only borrow and return change it.

```
class Equipment {
public:
    virtual std::string getType() const = 0;
};

class Microscope : public Equipment {
    std::string getType() const override
    { return "Microscope"; }
};
```

Inheritance and abstraction

One base, three children. The pure virtual method forces each child to answer.

```
std::vector<Equipment*> stock;
stock.push_back(new Microscope("CX23", "Olympus", 1000));

for (Equipment* e : stock)
    std::cout << e->detailLine();  // child line
```

Polymorphism

A base pointer calls the child version. The loop never checks the type.

```
Microscope(const std::string& n, const std::string& b, int mag);
Microscope(const std::string& n, const std::string& b);

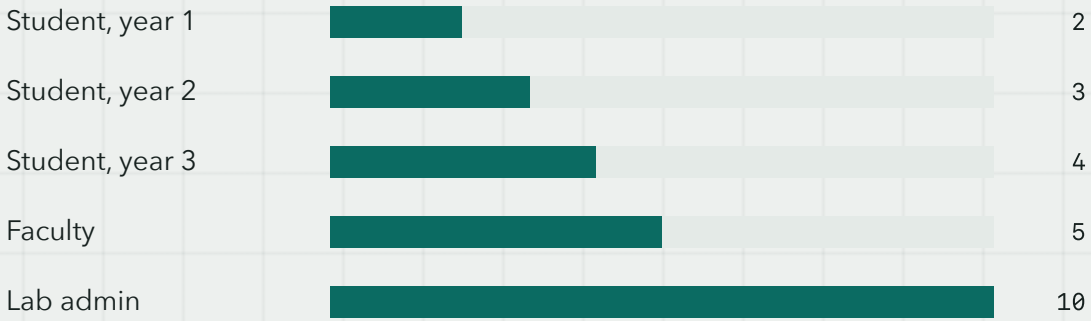
void search(const std::string& word);
void search(int id);

// Microscope("CX23", "Olympus")       400x
// Microscope("CX23", "Olympus", 1000) 1000x
```

Overloading

Same name, different inputs. The compiler picks one before the run.

Borrow limits depend on role. Fines depend on days late.



Items a person may hold at one time.

Loan length: 7 days for students, 14 for faculty, 30 for the admin.

A student borrows on day 1. The item is due on day 8.



10
STUDENT, PER DAY

5
FACULTY, PER DAY

0
LAB ADMIN

ITEM ALREADY OUT

Borrow fails and names the holder.

QUOTA REACHED

Borrow fails and states the limit.

REMOVE WHILE OUT

The item has to come back first.

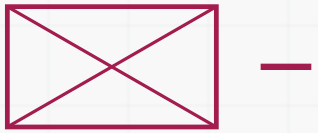
Three states the app shows.

Free



The item sits on the shelf. Anyone under their limit can take it.

Taken



The app names the holder. A second borrow of the same item is refused.

Late



The return is accepted. The app shows the days late and the amount owed.

The demo also shows a search by word and by id, and an admin adding a new instrument.

What it does not do.

Nothing is saved. Items, people and bookings live in memory and reset when the program closes.

Dates are counted by hand. The day moves with one menu key, so there are no real dates.

One person at a time. It runs in a terminal, with no logins across machines.

NEXT

Save the register to a file, and replace the day counter with real dates.

BUILT WITH

C++17 and the standard library only. No database.

CODE

github.com/vee1e/lab-booking-system

Questions